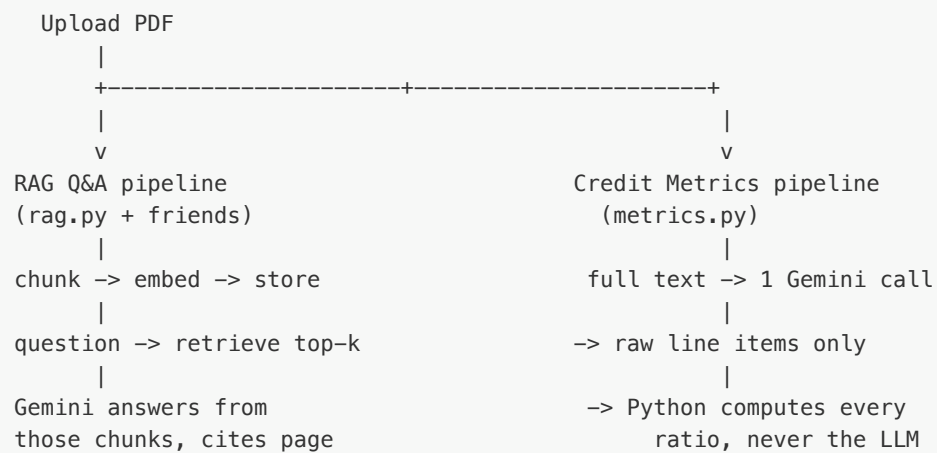


Credit Risk Analyzer

Code Walkthrough — RAG Q&A and Long-Context Credit Metrics



Repository: [credit-risk-analyzer](#) · Generated as a project reference document

1. Big picture

The app is two independent pipelines sharing one Gradio UI: a **RAG Q&A path** (the original feature) and a **long-context Credit Metrics path**. They share the PDF upload but never share code — deliberately, because they solve different problems.

Why two paths instead of one? Q&A needs *some* relevant passages for an arbitrary free-text question — retrieval is the right tool for that. Metrics need *all* the relevant numbers visible to the model at once (you can't retrieve-top-k your way to "every debt line item on the balance sheet"), and indexing a whole filing just to pull ~30 numbers would burn the free-tier embedding rate limit for no benefit. So metrics gets its own, simpler, one-shot extraction path instead of reusing RAG.

2. RAG Q&A path (pre-existing)

This pipeline answers free-text questions grounded in the uploaded filing, with page citations.

File	Job
config.py	Every tunable knob (models, chunk size, top-k, prompts) lives in one place, each overridable via an environment variable.
ingest.py	<code>pypdf</code> reads each page's text, then splits it into overlapping chunks. Every chunk is stamped with its source page number at parse time.
embeddings.py	A local <code>sentence-transformers</code> model turns chunks into vectors — no embedding API call, no per-token cost. Vectors are cached to disk keyed by a hash of the source bytes + config, so re-indexing the same PDF is instant.
vectorstore.py	Brute-force cosine similarity over an in-memory numpy matrix. Fine at single-document scale; deliberately a thin seam that can be swapped for FAISS/Chroma/pgvector later.
retry.py	Wraps any Gemini API call. Retries on 429 (rate limit) and 5xx using the server's suggested delay, or exponential backoff if none is given. A 429 with <i>no</i> suggested delay is treated as a hard quota exhaustion and fails immediately rather than retrying uselessly.
rag.py	Orchestration. <code>build_index()</code> runs ingest → embed → store once per upload. <code>answer_question()</code> embeds the question, retrieves the top-k most similar chunks, assembles a page-tagged prompt, and asks Gemini to answer <i>only</i> from that context.

The throughline that makes citations possible: a **page number** is stamped onto a chunk the moment it's parsed (`ingest.py`), and that tag rides unbroken through embedding, retrieval, and into the final prompt

as a literal `[p. 12] ...` marker — so the model can cite it, and a reader can verify it against the source document.

3. Credit Metrics path (new)

This is the feature added on top of the RAG app: extract raw financial line items from a 10-K and compute a full set of credit ratios from them — profitability, leverage, coverage, liquidity, cash flow, capital return, and the debt maturity schedule — each with a page-level citation, displayed as dashboard tiles.

3.1 metrics.py — the core module

Data model

Four dataclasses carry data from raw filing text through to a rendered tile:

- `LineItem` — one row, as-reported number: `value, unit, fiscal_year, source_statement, page`
- `MaturityYear` — one row of the debt schedule: a future calendar year mapped to principal due
- `ExtractedFinancials` — everything pulled from the filing: `fiscal_years, line_items: dict[name][year] -> LineItem`, and `debt_maturity_schedule`
- `DerivedMetric` — a *computed* ratio: `value, unit, formula, citations, reason` (why it's null, if it is), and `estimated` (whether the value relies on a labeled approximation)

Extraction — one Gemini call, no math allowed

1. `build_document_text()` tags every page `[p. N]` and concatenates the whole filing into one string — the same idea as the RAG path's context formatting, just applied to the entire document instead of a top-k slice.
2. `build_extraction_prompt()` lists the ~30 line items to pull, and gives the model a strict JSON schema to fill in. The prompt explicitly forbids computing anything — the model only reads off numbers and the page they came from.
3. `extract_line_items()` sends that prompt through `call_with_retry` (the same retry helper the RAG path uses), then hands the raw text response to `parse_llm_response()`, which strips ````json` code fences defensively and fills in any key the model omitted as an empty mapping rather than raising.

The LLM extracts, Python computes. This is the central design decision of the feature: a wrong number in a credit tool is worse than no number, and LLMs are unreliable at arithmetic. So the model's only job is reading off what's printed on the page; every ratio is calculated in plain Python where it can be tested and trusted.

Computation — pure Python

Six group functions compute the actual metrics: `compute_profitability`, `compute_leverage`, `compute_coverage`, `compute_liquidity`, `compute_cash_flow`, `compute_capital_return`. All six are built on one shared helper:

```
_combine(name, formula, fiscal_year, inputs, fn, unit)
```

`_combine` checks every input before doing any arithmetic:

Strict null propagation, never a zero substitution. If any input is missing (`None` or has no value), the whole metric becomes `None` with a human-readable `reason` string — it is never computed by silently treating a missing figure as zero. This cascades: a missing `commercial_paper` line nulls out `total_debt`, which then nulls `net_debt`, which then nulls `net_debt_to_ebitda`, `FF0/debt`, and `FCF/debt` — the "unavailable" state propagates through the entire dependency graph rather than quietly understating leverage. `compute_all_derived_metrics()` wires the six groups together, threading computed values like `total_debt`, `ebitda`, and `free_cash_flow` in as inputs to the metrics that depend on them.

Two situations got hand-written special cases instead of generic-helper treatment:

- **Netted interest expense** (`compute_coverage`) — many filings (Apple's included) don't report interest expense as its own line; it's netted inside "Other income/(expense), net." When that happens, coverage falls back to a labeled estimate — interest payable within 12 months, read from the debt-maturity note — and marks the result `estimated=True` with a reason explaining the substitution, so the UI can show the caveat instead of a falsely precise number.
- **Net-cash rendering** (`render_metric_tile_html`) — a negative `net_debt` is a *good* sign (more cash than debt), so instead of displaying "\$-33.8B" the tile shows "Net cash: \$33.8B", and a negative `net_debt_to_ebitda` shows "N/M (net cash position)" rather than a meaningless negative leverage multiple.

Rendering

`render_dashboard_html()` groups the six metric dictionaries into styled HTML tiles (inline CSS, no JS dependency) with a citation line per tile — e.g. "Balance Sheet, p.35" — and a caveat line for anything estimated or unavailable.

3.2 app.py — Gradio wiring

The whole file is four handler functions plus one `gr.Blocks()` layout.

Handler	Role
<code>process_pdf</code>	Unchanged. Builds the RAG index on upload, shows an indexing progress bar.
<code>ask</code>	Unchanged. Answers a question from the RAG index and appends page citations.
<code>process_metrics</code>	The new handler. Calls <code>extract_line_items</code> with an <code>on_retry</code> callback that updates the Gradio progress bar during a rate-limit backoff — so a retry reads "retrying in 21s..." instead of the UI looking frozen. Returns a 4th value, a <code>gr.Button(interactive=...)</code> , that disables the extract button on success and leaves it enabled on any failure so the user can retry.
<code>reset_metrics_ui</code>	Fires whenever a new PDF is uploaded. Clears the stale dashboard/chart/status and re-enables the extract button, since old metrics belong to the previous document.

Layout: one shared file uploader above two tabs — *Ask Questions* and *Credit Metrics* — reflecting the single-document-per-session scope of the app. The `show_progress_on=metrics_status` argument on the metrics button's `.click()` pins Gradio's progress overlay to just that one status component, instead of duplicating the same progress text across every output of the event.

4. Tests — tests/test_metrics.py

The derived-metrics tests never touch the network. They build an `ExtractedFinancials` from a hand-written dictionary of **real Apple FY2025/FY2024 figures** — pulled directly from the actual fixture PDF's pages — and assert the ratio functions reproduce known targets:

Metric	Target
Gross margin	46.9%
EBITDA	\$144.7B
Net debt	−\$33.8B (net cash)
Current ratio	0.89x

Separate tests cover the edge cases directly and deterministically:

- The netted-interest fallback engaging correctly and flagging `estimated=True`
- A missing debt component nulling `total_debt` and every metric downstream of it
- Division by zero returning `None` + a reason instead of raising
- Defensive JSON parsing correctly stripping markdown code fences

One test mocks the Gemini client (a fake object standing in for `.models.generate_content`) to exercise `extract_line_items` end-to-end without ever hitting the network — so the whole suite runs with no API key.

5. Where things live

```
credit-risk-analyzer/
├── app.py
├── src/credit_risk_analyzer/
│   ├── config.py
│   ├── ingest.py
│   ├── embeddings.py
│   ├── vectorstore.py
│   ├── retry.py
│   ├── rag.py
│   └── metrics.py
├── tests/
│   ├── test_ingest.py
│   ├── test_embeddings.py
│   ├── test_retry.py
│   ├── test_metrics.py
│   └── fixtures/apple-10k-2025.pdf
└── docs/
    ├── architecture.md
    └── REQUIREMENTS.md
```

Gradio UI: both tabs, all event wiring

Models, chunk size, top-k, prompts

PDF -> page text -> page-tagged chunks

Local embedding model + disk cache

In-memory cosine-similarity search

Shared Gemini call retry/backoff helper

RAG orchestration: index + grounded Q&A

Long-context extraction + Python ratio engine

Fixed real-figure fixture, no API key needed

System design, both pipelines diagrammed

FR/NFR requirements, including FR-16..FR-20